Hashcat 연산 검증

송민호

유튜브:https://youtu.be/VuGcrSjm8kM

Hashcat

- 정상 작동 결과

```
smino@songminhos-MacBook-Air hashcat % ./hashcat -m 0 -a 6 hash.txt candidates.txt '
?d?d'
hashcat (v7.1.2-382-g2d71af371) starting

METAL API (Metal 306.7.5)
=========================
* Device #01: Apple M2, skipped

OpenCL API (OpenCL 1.2 (Jun 23 2023 20:24:12)) - Platform #1 [Apple]
====================================================================
* Device #02: Apple M2, GPU, 2730/5461 MB (512 MB allocatable), 8MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 100c

Host memory allocated for this attack: 650 MB (1474 MB free)
```

```
Dictionary cache hit:
* Filename..: candidates.txt
* Passwords.: 1
* Bytes.....: 9
* Keyspace..: 100

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Hashcat is expecting at least 458752 base words but only got 0.0% of that.
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

08f5b04545cbf7eaa238621b9ab84734:Password12

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 08f5b04545cbf7eaa238621b9ab84734
Time.Started.....: Tue Apr 14 15:57:32 2026 (0 secs)
Time.Estimated...: Tue Apr 14 15:57:32 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (candidates.txt), Left Side
Guess.Mod........: Mask (?d?d) [2], Right Side
Guess.Queue.Base.: 1/1 (100.00%)
Guess.Queue.Mod..: 1/1 (100.00%)
Speed.#02........:     8749 H/s (0.00ms) @ Accel:224 Loops:64 Thr:256 Vec:1
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 64/100 (64.00%)
Rejected.........: 0/64 (0.00%)
Restore.Point....: 0/1 (0.00%)
Restore.Sub.#02..: Salt:0 Amplifier:0-64 Iteration:0-64
Candidate.Engine.: Device Generator
Candidates.#02...: Password12 -> Password34
Hardware.Mon.#02.: Util: 82% Pwr:975mW

Started: Tue Apr 14 15:57:30 2026
Stopped: Tue Apr 14 15:57:33 2026
```

Self-test

- Self-test는 hashcat/src/selftest.c에서 진행
- Self-test는 본 작업을 시작하기 전에 디바이스가 계산을 똑바로 수행하는지 점검하는 과정

- Hashcat 실행 시 Self-test 진행
- KAT을 통해 Self-test 진행
  - ST_PASS, ST_HASH

- MD5 예시
  - Hashcat/src/modules/module_00000.c

p1 > hashcat > src > modules > C module_00000.c

```
12
13  static const u32   ATTACK_EXEC    = ATTACK_EXEC_INSIDE_KERNEL;
14  static const u32   DGST_POS0      = 0;
15  static const u32   DGST_POS1      = 3;
16  static const u32   DGST_POS2      = 2;
17  static const u32   DGST_POS3      = 1;
18  static const u32   DGST_SIZE      = DGST_SIZE_4_4;
19  static const u32   HASH_CATEGORY  = HASH_CATEGORY_RAW_HASH;
20  static const char *HASH_NAME      = "MD5";
21  static const u64   KERN_TYPE      = 0;
22  static const u32   OPTI_TYPE      = OPTI_TYPE_ZERO_BYTE
23                                    | OPTI_TYPE_PRECOMPUTE_INIT
24                                    | OPTI_TYPE_MEET_IN_MIDDLE
25                                    | OPTI_TYPE_EARLY_SKIP
26                                    | OPTI_TYPE_NOT_ITERATED
27                                    | OPTI_TYPE_NOT_SALTED
28                                    | OPTI_TYPE_RAW_HASH;
29  static const u64   OPTS_TYPE      = OPTS_TYPE_STOCK_MODULE
30                                    | OPTS_TYPE_PT_GENERATE_LE
31                                    | OPTS_TYPE_PT_ADD80
32                                    | OPTS_TYPE_PT_ADDBITS14;
33  static const u32   SALT_TYPE      = SALT_TYPE_NONE;
34  static const char *ST_PASS        = "hashcat";
35  static const char *ST_HASH        = "8743b52063cd84097a65d1633f5c74f5";
```

Self-test 과정

- Process_selftest를 통해 init, run_kernel, cleanup 순서대로 진행

- init은 self-test용으로 상태를 설정
  - 실제 해시 목록 대신 self-test용 해시 설정

- run_kernel은 실제와 같은 커널 실행

- cleanup은 결과를 읽음
  - 이후 설정 초기화(버퍼 등)

```
static int process_selftest (hashcat_ctx_t *hashcat_ctx, hc_device_param_t *device_param)
{
  hashconfig_t *hashconfig = hashcat_ctx->hashconfig;
  status_ctx_t *status_ctx = hashcat_ctx->status_ctx;

  if (hashconfig->st_hash == NULL) return 0;

  u32 highest_pw_len = 0;
  u32 num_cracked = 0;

  if (selftest_init (hashcat_ctx, device_param, &highest_pw_len) == -1) return -1;

  if (selftest_run_kernel (hashcat_ctx, device_param, highest_pw_len) == -1) return -1;

  if (selftest_cleanup (hashcat_ctx, device_param, &num_cracked) == -1) return -1;
```

Self-test 결과

- Self-test 실패시 (해시값 형식이 틀린 경우)
  Self-test hash parsing error 뜨고 hashcat 중단

p1 > hashcat > src > modules > C module_00000.c

```
12
13    static const u32   ATTACK_EXEC    = ATTACK_EXEC_INSIDE_KERNEL;
14    static const u32   DGST_POS0      = 0;
15    static const u32   DGST_POS1      = 3;
16    static const u32   DGST_POS2      = 2;
17    static const u32   DGST_POS3      = 1;
18    static const u32   DGST_SIZE      = DGST_SIZE_4_4;
19    static const u32   HASH_CATEGORY  = HASH_CATEGORY_RAW_HASH;
20    static const char *HASH_NAME      = "MD5";
21    static const u64   KERN_TYPE      = 0;
22    static const u32   OPTI_TYPE      = OPTI_TYPE_ZERO_BYTE
23                                      | OPTI_TYPE_PRECOMPUTE_INIT
24                                      | OPTI_TYPE_MEET_IN_MIDDLE
25                                      | OPTI_TYPE_EARLY_SKIP
26                                      | OPTI_TYPE_NOT_ITERATED
27                                      | OPTI_TYPE_NOT_SALTED
28                                      | OPTI_TYPE_RAW_HASH;
29    static const u64   OPTS_TYPE      = OPTS_TYPE_STOCK_MODULE
30                                      | OPTS_TYPE_PT_GENERATE_LE
31                                      | OPTS_TYPE_PT_ADD80
32                                      | OPTS_TYPE_PT_ADDBITS14;
33    static const u32   SALT_TYPE      = SALT_TYPE_NONE;
34    static const char *ST_PASS        = "hashcat";
35    static const char *ST_HASH        = "8743b52063cd84097a65d1633f5c74f5";
```

```
34    static const char *ST_PASS        = "hashcat";
35    static const char *ST_HASH        = "8743b52063cd84097a65d1633f5c74f5.";
```

```
smino@songminhos-MacBook-Air hashcat % ./hashcat -m 0 -a 6 hash.txt candidates.txt '
?d?d'
hashcat (v7.1.2-382-g2d71af371+) starting

METAL API (Metal 306.7.5)
=========================
* Device #01: Apple M2, skipped

OpenCL API (OpenCL 1.2 (Jun 23 2023 20:24:12)) - Platform #1 [Apple]
====================================================================
* Device #02: Apple M2, GPU, 2730/5461 MB (512 MB allocatable), 8MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Self-test hash parsing error: Token length exception

Started: Tue Apr 14 15:06:05 2026
Stopped: Tue Apr 14 15:06:06 2026
```

Self-test 결과

- Self-test 실패시(해시값이 일치하지 않은 경우)
  Self-test failed 뜨고 hashcat 중단

p1 › hashcat › src › modules › C module_00000.c

```
12
13    static const u32    ATTACK_EXEC     = ATTACK_EXEC_INSIDE_KERNEL;
14    static const u32    DGST_POS0       = 0;
15    static const u32    DGST_POS1       = 3;
16    static const u32    DGST_POS2       = 2;
17    static const u32    DGST_POS3       = 1;
18    static const u32    DGST_SIZE       = DGST_SIZE_4_4;
19    static const u32    HASH_CATEGORY   = HASH_CATEGORY_RAW_HASH;
20    static const char *HASH_NAME        = "MD5";
21    static const u64    KERN_TYPE       = 0;
22    static const u32    OPTI_TYPE       = OPTI_TYPE_ZERO_BYTE
23                                        | OPTI_TYPE_PRECOMPUTE_INIT
24                                        | OPTI_TYPE_MEET_IN_MIDDLE
25                                        | OPTI_TYPE_EARLY_SKIP
26                                        | OPTI_TYPE_NOT_ITERATED
27                                        | OPTI_TYPE_NOT_SALTED
28                                        | OPTI_TYPE_RAW_HASH;
29    static const u64    OPTS_TYPE       = OPTS_TYPE_STOCK_MODULE
30                                        | OPTS_TYPE_PT_GENERATE_LE
31                                        | OPTS_TYPE_PT_ADD80
32                                        | OPTS_TYPE_PT_ADDBITS14;
33    static const u32    SALT_TYPE       = SALT_TYPE_NONE;
34    static const char *ST_PASS          = "hashcat";
35    static const char *ST_HASH          = "8743b52063cd84097a65d1633f5c74f5";
```

```
34    static const char *ST_PASS          = "hashcat";
35    static const char *ST_HASH          = "8743b52063cd84097a65d1633f5c74f6";
```

```
Watchdog: Temperature abort trigger set to 100c

Host memory allocated for this attack: 650 MB (1596 MB free)

* Device #2: ATTENTION! OpenCL kernel self-test failed.

Your device driver installation is probably broken.
See also: https://hashcat.net/faq/wrongdriver

Aborting session due to kernel self-test failure.

You can use --self-test-disable to override, but do not report related errors.

Started: Tue Apr 14 16:38:09 2026
Stopped: Tue Apr 14 16:38:10 2026
```

Chech_cracked

- Crack의 결과는 hashcat/src/hashes.c에서 진행
- 후보 비밀번호마다 해시를 계산해서 내가 넣은 해시값과 같은지 확인
  - 디바이스에서 진행되며 값을 찾으면 num_cracked에 결과 개수를 기록

- check_cracked는 디바이스의 결과를 확인

```
//int check_cracked (hashcat_ctx_t *hashcat_ctx, hc_device_param_t *device_param,
int check_cracked (hashcat_ctx_t *hashcat_ctx, hc_device_param_t *device_param)
{
  cpt_ctx_t      *cpt_ctx      = hashcat_ctx->cpt_ctx;
  hashconfig_t   *hashconfig   = hashcat_ctx->hashconfig;
  hashes_t       *hashes       = hashcat_ctx->hashes;
  status_ctx_t   *status_ctx   = hashcat_ctx->status_ctx;
  user_options_t *user_options = hashcat_ctx->user_options;

  u32 num_cracked = 0;
```

Chech_hash

- Crack이라면 check_hash 진행

- 디바이스가 넘긴 크랙 정보로 호스트에서 해시 문자열과 평문을 조립
  - 이후 potfile이나 아웃파일에 저장 후 터미널에 표시

```
int check_hash (hashcat_ctx_t *hashcat_ctx, hc_device_param_t *device_param, plain_t *plain)
{
  const debugfile_ctx_t *debugfile_ctx = hashcat_ctx->debugfile_ctx;
  const hashes_t        *hashes        = hashcat_ctx->hashes;
  const hashconfig_t    *hashconfig    = hashcat_ctx->hashconfig;
  const loopback_ctx_t  *loopback_ctx  = hashcat_ctx->loopback_ctx;
  const module_ctx_t    *module_ctx    = hashcat_ctx->module_ctx;

  const u32 salt_pos    = plain->salt_pos;
  const u32 digest_pos  = plain->digest_pos;  // relative

  void *tmps = NULL;
```

Q & A